

General Relativity in the $\mathcal{UD}$ Calculus

Brian Beckman

Jan 2026

Abstract

In an [earlier notebook](#), I validated the $\mathcal{UD}$ (up-down) calculus against Carlip’s electrodynamical aberration. In this notebook, I turn to the second and motivating problem in [Carlip’s paper, Aberration and the Speed of Gravity](#): gravitational aberration.

The $\mathcal{UD}$ calculus is a collection of rules and operators for both unambiguous input and readable output of tensor index notations. It’s becoming an [embedded domain-specific language \(eDSL\)](#).

The electrodynamical case required only special relativity, and was thus a bit smaller than the gravitational case. Here, we use the same $\mathcal{UD}$ to calculate Christoffel connections and the Riemann curvature tensor in a weak-field approximation. These yield a similar aberration effect in gravitation to that seen in electrodynamics: the velocity of a moving source causes the apparent direction of gravitational influence to point approximately to the instantaneous position of the source rather than to its retarded position. In the electrodynamical case, the approximation is linear in the Lorentz-invariant 4-velocity. In the gravitational case, the approximation is better: it’s quadratic in the velocity and exact if the source is accelerating but not jerking.

At first glance, aberration seems absurd. How can a particle “know” where the source is “now” when “now” can’t even be defined invariantly, that is, without choosing a coordinate system? But, in retrospect, it has to be this way. The retarded 3-position of a moving source—where it was when the gravitational influence was emitted—is not Lorentz-invariant. Thus *it* also depends on choice of coordinates. But no choice of coordinate can affect the physics. A test particle cannot “know where the source was” any more than it can know where it is “now.” Instead, the source sees a sum of Lorentz-invariant quantities involving the invariant retarded distance and its derivatives: velocity and acceleration, etc. This sum resembles “knowledge,” but it is just an extrapolation of Lorentz-invariant quantities, however, resolving the conundrum.

This notebook, like its predecessor, is more about computation in $\mathcal{UD}$ than it is about physics. To keep this notebook short, we have only cursory, philosophical remarks about physical interpretation. We refer you to the original paper and its references for more crisp discussion of the physics.

Introduction

In general relativity, the “force” is not a vector field but a curvature of spacetime, encoded in the

Riemann Tensor $R^\rho_{\sigma\mu\nu}$. To investigate aberration, we must derive this curvature for a moving source. Standard manual derivations of Riemann are notoriously error-prone due to the proliferation of indices. Here, we apply $\mathcal{UD}$'s `fourGrad` operator to derive indexed objects symbolically from their definitions. The resulting formal verification easily justifies the effort of developing $\mathcal{UD}$, as checking and double checking a manual calculation would have taken much longer.

The $\mathcal{UD}$ Implementation

Here, we “paste” the implementation without commentary, referring you to the [earlier notebook](#) for walkthrough. $\mathcal{UD}$ was developed for the easier electrodynamical case, but ended up working well here, without modification.

- *This section contains mostly general-purpose rules for $\mathcal{UD}$, but there are special rules in the mix, such as those for invariant retarded distance and some rules for `fourGrad`. A clean separation of general from special awaits more meticulous engineering of $\mathcal{UD}$.*

Display Rules

```
In[*]:= ClearAll[dpyRules, d, s, λ, σ, η, dλds];
dpyRules = {
  (*Specific rules ahead of general rules*)
  dλds[ $\mathcal{D}[\nu\_]$ ]  $\Rightarrow$  (FractionBox[d Subscript[λ, ν], d s] // DisplayForm),
  dλds[ $\mathcal{U}[\nu\_]$ ]  $\Rightarrow$  (FractionBox[d Superscript[λ, ν], d s] // DisplayForm),
  (*General Rules*)
  ε[ $\mathcal{U}[\tau\_]$ ]  $\Rightarrow$  Superscript[ε, τ],
  ε[ $\mathcal{D}[\tau\_]$ ]  $\Rightarrow$  Subscript[ε, τ],
  ε[ $\mathcal{D}[\mu\_]$ ,  $\mathcal{D}[\nu\_]$ ]  $\Rightarrow$  Subscript[ε, Row[{μ, ν}]],
  ε[ $\mathcal{D}[\mu\_]$ ,  $\mathcal{U}[\nu\_]$ ]  $\Rightarrow$  Superscript[Subscript[ε, μ], ν],
  ε[ $\mathcal{U}[\mu\_]$ ,  $\mathcal{U}[\nu\_]$ ]  $\Rightarrow$  Superscript[ε, Row[{μ, ν}]],
  ε[ $\mathcal{U}[\mu\_]$ ,  $\mathcal{D}[\nu\_]$ ]  $\Rightarrow$  Subscript[Superscript[ε, μ], ν]
};
ClearAll[echo]; (*for debugging*)
echo[τ_] := Module[{},
  Print[τ /. dpyRules];
  Echo[τ];
];
```

Tensor Rules

```

In[*]:= ClearAll[tensorRules];
tensorRules = {
  (*Special Rules First*)
  (*Definition of  $\sigma$ *)
  x[ud_] - z[s][ud_]  $\Rightarrow$   $\sigma$ [ud],
  (*Null Interval*)
   $\sigma[\mathcal{U}[\alpha_]] \times \sigma[\mathcal{D}[\alpha_]] \Rightarrow 0$ ,
   $\sigma[\mathcal{D}[\alpha_]] \times \sigma[\mathcal{U}[\alpha_]] \Rightarrow 0$ ,
  (*Definition of  $\lambda$ *)
  z'[s][ud_]  $\Rightarrow$   $\lambda$ [ud],
  (*Unit Velocity*)
   $\lambda[\mathcal{U}[\alpha_]] \times \lambda[\mathcal{D}[\alpha_]] \Rightarrow 1$ ,
   $\lambda[\mathcal{D}[\alpha_]] \times \lambda[\mathcal{U}[\alpha_]] \Rightarrow 1$ ,
  (*Mixed Metric  $\eta_{\mu}^{\rho}$  acts as  $\delta_{\mu}^{\rho}$ *)
   $\eta[\mathcal{D}[\mu_], \mathcal{U}[\rho_]] \Rightarrow \delta[\mathcal{D}[\mu], \mathcal{U}[\rho]]$ ,
  (*Metric Symmetry, avoiding unbounded recursion via OrderedQ*)
   $\eta[\mathcal{D}[\alpha_], \mathcal{D}[\beta_]] /; \text{OrderedQ}[\{\alpha, \beta\}] \Rightarrow \eta[\mathcal{D}[\beta], \mathcal{D}[\alpha]]$ ,
  (*General Rules*)
  (*Coalesce Terms into Down-Up Order*)
   $\tau_{-}[\mathcal{D}[\mu_]] \times \nu_{-}[\mathcal{U}[\mu_]] /; \text{OrderedQ}[\{\tau, \nu\}] \Rightarrow \nu[\mathcal{D}[\mu]] \times \tau[\mathcal{U}[\mu]]$ ,
  (*Raising and Lowering: e.g.  $\eta_{\mu\alpha}\lambda^{\alpha} \rightarrow \lambda_{\mu}$ *)
   $\eta[\mathcal{D}[\mu_], \mathcal{D}[\nu_]] \times \tau_{-}[\mathcal{U}[\nu_]] \Rightarrow \tau[\mathcal{D}[\mu]]$ ,
   $\eta[\mathcal{D}[\mu_], \mathcal{D}[\nu_]] \times \tau_{-}[\mathcal{U}[\mu_]] \Rightarrow \tau[\mathcal{D}[\nu]]$ ,
   $\eta[\mathcal{D}[\mu_], \mathcal{D}[\nu_]] (\tau_{-} + \nu_{-}) \Rightarrow \eta[\mathcal{D}[\mu], \mathcal{D}[\nu]] \tau + \eta[\mathcal{D}[\mu], \mathcal{D}[\nu]] \nu$ ,
   $\eta[\mathcal{D}[\mu_], \mathcal{D}[\nu_]] (\tau_{-}[\mathcal{U}[\nu_]] \times \nu_{-}[\mathcal{U}[\mu_]]) \Rightarrow \tau[\mathcal{D}[\mu]] \times \nu[\mathcal{U}[\mu]]$ ,
  (*unraveling recursive  $\mathcal{UD}$  indices*)
   $\tau_{-}[(h1 : \mathcal{U} | \mathcal{D})[(h2 : \mathcal{U} | \mathcal{D})[ud_]]] /; h1 == h2 \Rightarrow \tau[h1[ud]]$ 
};

```

Invariant Retarded Distance r

```

In[*]:= ClearAll[rExpr];
(rExpr =  $\sigma[\mathcal{D}[\rho]] \times \lambda[\mathcal{U}[\rho]]$ ) /. dpyRules
Out[*]=
 $\sigma_{\rho} \lambda^{\rho}$ 

```

Custom 4-grad Operator

Basic Rules from Calculus

Linearity, Product Rule, Power Rule

```
In[*]:= fourGrad[a_ + b_, ud_] := fourGrad[a, ud] + fourGrad[b, ud];
fourGrad[a_ * b_, ud_] := b fourGrad[a, ud] + a fourGrad[b, ud];
fourGrad[a_^n_, ud_] := n a^(n-1) fourGrad[a, ud];
```

Constants e and k

```
In[*]:= fourGrad[e, _] := 0;
fourGrad[k, _] := 0;
```

Numeric Constants

```
In[*]:= fourGrad[n_?NumericQ, _] := 0;
```

Gradient of the Separation Vector σ

```
In[*]:= fourGrad[sigma[udv_], udmu_] := eta[udmu, udv] - lambda[udv]  $\frac{\sigma[udmu]}{r}$ ;
```

```
In[*]:= fourGrad[sigma[D[v]], D[mu]] /. dpyRules
```

```
Out[*]= 
$$\eta_{\mu\nu} - \frac{\lambda_\nu \sigma_\mu}{r}$$

```

Gradient of the Velocity λ

```
In[*]:= fourGrad[lambda[udv_], udmu_] := dlambda[udv]  $\frac{\sigma[udmu]}{r}$ ;
```

```
In[*]:= fourGrad[lambda[D[v]], D[mu]] /. dpyRules
```

```
Out[*]= 
$$\frac{\frac{d\lambda_\nu}{ds} \sigma_\mu}{r}$$

```

$\partial_\mu r$: Gradient of r

```
In[*]:= (gradRRaw = fourGrad[rExpr, D[mu]]) /. dpyRules
```

```
Out[*]= 
$$\frac{\frac{d\lambda^\rho}{ds} \sigma_\mu \sigma_\rho}{r} + \left( \eta_{\mu\rho} - \frac{\lambda_\rho \sigma_\mu}{r} \right) \lambda^\rho$$

```

```
In[*]:= (fourGrad[r, mu_] = Simplify[gradRRaw //. tensorRules] //. tensorRules) /. dpyRules
```

```
Out[*]= 
$$\lambda_\mu + \frac{\sigma_\mu \left( -1 + \frac{d\lambda^\rho}{ds} \sigma_\rho \right)}{r}$$

```

The Physics

The Metric Perturbation $h_{\mu\nu}$

In the weak-field approximation (linearized gravity), the metric $g_{\mu\nu}$ is the Minkowski metric $\eta_{\mu\nu}$ plus a small perturbation $h_{\mu\nu}$:

$$g_{\mu\nu} \approx \eta_{\mu\nu} + h_{\mu\nu} \quad (1)$$

For a point mass M moving with 4-velocity λ_μ , the metric perturbation is the direct gravitational analog of the electrodynamical Liénard-Wiechert potential. Carlip gives this in his Equation 2.1:

$$h_{\mu\nu} = \frac{4GM}{r} \lambda_\mu \lambda_\nu \quad (2)$$

In $\mathcal{UD}$, we capture the constant factor $4GM \equiv k$ as a symbolic constant k without a value, and write

```
In[*]:= ClearAll[h, k, metricGradient, gradHRaw];
(*Gradient of the constant k*)
fourGrad[k, _] := 0;
(*Perturbation tensor*)
h[D[μ_], D[ν_]] :=  $\frac{k}{r} \lambda[D[\mu]] \times \lambda[D[\nu]]$ ;
(*Display rule for h*)
AppendTo[dpyRules, h[D[μ_], D[ν_]]  $\Rightarrow$  Subscript[h, Row[{μ, ν}]]];
(*Verify the display*)
h[D[μ], D[ν]] /. dpyRules

Out[*]=
 $h_{\mu\nu}$ 
```

Gradient $\partial_\rho h_{\mu\nu}$ of the Metric Perturbation $h_{\mu\nu}$

Invoke the previously defined `rExpr` and the `fourGrad` rule for r via manually renamed summation index γ that does not collide with the free index ρ .

■ This kind of collision will be fixed by “alpha renaming” if and when we write a proper compiler for $\mathcal{UD}$.

```
In[*]:= gradHRaw = fourGrad[h[D[μ], D[ν]], D[γ]];
(*Simplify and apply tensor rules*)
metricGradient = Collect[Expand[gradHRaw /. tensorRules], {r, k}, FullSimplify];

Print["Gradient  $\partial_\rho h_{\mu\nu}$  of the metric perturbation  $h_{\mu\nu}$ :"];
metricGradient /. dpyRules
```

Gradient $\partial_\rho h_{\mu\nu}$ of the metric perturbation $h_{\mu\nu}$:

```
Out[*]=
```

$$\frac{k \left(-\lambda_\gamma \lambda_\mu \lambda_\nu + \left(\frac{d\lambda_\nu}{ds} \lambda_\mu + \frac{d\lambda_\mu}{ds} \lambda_\nu \right) \sigma_\gamma \right)}{r^2} + \frac{k \lambda_\mu \lambda_\nu \sigma_\gamma \left(1 - \frac{d\lambda^\rho}{ds} \sigma_\rho \right)}{r^3}$$

Intuitions

The second term, with r^3 in the denominator, is Coulomb-like, scaling as $1/r^2$ because the σ in the numerator represents the physical distance between source and test and scales like r . The first term, with r^2 in the denominator, is radiative, scaling partly as $1/r$ due also to a σ in the denominator, but also scaling with the acceleration $d\lambda/ds$. A field proportional to both acceleration and to $1/r$ is the definition of a radiative field.

The factor $1 - \frac{d\lambda^0}{ds} \sigma_\rho$ in the Coulomb-like portion of the field accounts for changes in the gradient of r due to acceleration $\frac{d\lambda}{ds}$. It encapsulates the quadratic part of the extrapolation of the retarded position, schematically $x_{\text{ret}} + v\Delta t + \frac{1}{2}a\Delta t^2$. The velocity of the source causes level surfaces of the Coulomb field to “squish” in front of the source, a **Doppler-like effect**, and acceleration causes the spacing of the surfaces to compress.

The Christoffel Connection $\Gamma^\tau_{\mu\nu}$

The “force” in gravitation arises from the non-tensorial connection, $\Gamma^\tau_{\mu\nu}$.

- *Aside: Gravitation shares this idea with quantum fields, where force bosons come from the connections of particular gauge fields that leave Lagrangians invariant. This similarity is so alluring that physicists strongly suspect an underlying unified theory. However, unification has resisted all attempts, making predictions refuted by experiment, failing renormalization, etc. It is an open question whether the unification even exists or whether Nature gives us two different but similar-looking kinds of force laws (space-time and gauge), laughing at our petty struggles to understand.*

In the weak field limit ($g_{\mu\nu} \approx \eta_{\mu\nu} + h_{\mu\nu}$), $\Gamma^\tau_{\mu\nu}$ is defined by linear combinations of ∂h :

$$\Gamma^\tau_{\mu\nu} \approx \frac{1}{2} \eta^{\tau\chi} (\partial_\mu h_{\nu\chi} + \partial_\nu h_{\mu\chi} - \partial_\chi h_{\mu\nu}) \quad (3)$$

To implement this efficiently in $\mathcal{UD}$, we define a helper function to “instantiate” `metricGradient` with specific indices.

```
In[*]:= ClearAll[gradH, rLinear];

In[*]:= (*Helper: instantiate  $\partial_\rho h_{\alpha\beta}$  with specific indices*)
gradH[x $\gamma$ _, x $\alpha$ _, x $\beta$ _] := metricGradient /. { $\mathcal{D}[\gamma] \rightarrow x\gamma$ ,  $\mathcal{D}[\mu] \rightarrow x\alpha$ ,  $\mathcal{D}[\nu] \rightarrow x\beta$ };
(*test*)
gradH[ $\mathcal{D}[\mu]$ ,  $\mathcal{D}[\nu]$ ,  $\mathcal{D}[\chi]$ ] /. dpyRules

Out[*]:= 
$$\frac{k \left( -\lambda_\mu \lambda_\nu \lambda_\chi + \left( \frac{d\lambda_\chi}{ds} \lambda_\nu + \frac{d\lambda_\nu}{ds} \lambda_\chi \right) \sigma_\mu \right)}{r^2} + \frac{k \lambda_\nu \lambda_\chi \sigma_\mu \left( 1 - \frac{d\lambda^0}{ds} \sigma_\rho \right)}{r^3}$$

```

```

In[*]:= (*leave index  $\chi$  free for later contraction *)
 $\Gamma$ Linear[ $\mathcal{U}[\tau\_]$ ,  $\mathcal{D}[\mu\_]$ ,  $\mathcal{D}[\nu\_]$ ] :=  $\frac{1}{2} \eta[\mathcal{U}[\tau], \mathcal{U}[\chi]]$ 
      (gradH[ $\mathcal{D}[\mu]$ ,  $\mathcal{D}[\nu]$ ,  $\mathcal{D}[\chi]$ ] + gradH[ $\mathcal{D}[\nu]$ ,  $\mathcal{D}[\mu]$ ,  $\mathcal{D}[\chi]$ ] - gradH[ $\mathcal{D}[\chi]$ ,  $\mathcal{D}[\mu]$ ,  $\mathcal{D}[\nu]$ ]);

(*Calculate a specific instance and display it*)
 $\Gamma$ Instance =  $\Gamma$ Linear[ $\mathcal{U}[\tau]$ ,  $\mathcal{D}[\mu]$ ,  $\mathcal{D}[\nu]$ ] /. tensorRules;
 $\Gamma$ Result = Collect[Expand[ $\Gamma$ Instance], {r, k}, FullSimplify];

Print["Linearized Christoffel connection  $\Gamma^{\tau}_{\mu\nu}$ :"];
( $\Gamma$ Result /. FullSimplify) /. dpyRules
Linearized Christoffel connection  $\Gamma^{\tau}_{\mu\nu}$ :

```

$$\begin{aligned}
\text{Out}[*]= & \frac{1}{2 r^3} k \left(r \left(\frac{d \lambda_{\chi}}{d s} \lambda_{\nu} \sigma_{\mu} + \lambda_{\chi} \left(\frac{d \lambda_{\nu}}{d s} \sigma_{\mu} + \frac{d \lambda_{\mu}}{d s} \sigma_{\nu} \right) - \frac{d \lambda_{\mu}}{d s} \lambda_{\nu} \sigma_{\chi} - \lambda_{\mu} \left(\lambda_{\nu} \lambda_{\chi} - \frac{d \lambda_{\chi}}{d s} \sigma_{\nu} + \frac{d \lambda_{\nu}}{d s} \sigma_{\chi} \right) \right) - \right. \\
& \left. \left(-1 + \frac{d \lambda^{\rho}}{d s} \sigma_{\rho} \right) (\lambda_{\mu} \lambda_{\chi} \sigma_{\nu} + \lambda_{\nu} (\lambda_{\chi} \sigma_{\mu} - \lambda_{\mu} \sigma_{\chi})) \right) \eta^{\tau\chi}
\end{aligned}$$

Intuitions

Γ encapsulates effects we can intuit as forces. The Geodesic Equation for a test particle with 4-velocity v^{μ} gives the equation of motion:

$$\frac{dv^x}{d\tau} = -\Gamma^x_{\mu\nu} v^{\mu} v^{\nu} \quad (4)$$

which resembles Newtonian $F = ma$, with a as $\frac{dv^x}{d\tau}$ and the negative gradient of some velocity-dependent potential $-\nabla\Phi$ as $-\Gamma^x_{\mu\nu} v^{\mu} v^{\nu}$. The terms in Γ that scale as $1/r^2$, namely the first term, represent the gravitational pull from the Coulomb-like part of the metric gradient. The “Doppler-like” factor in the second term ensures that the $1/r^2$ “force” points to the instantaneous position of the source!

The Riemann Curvature Tensor ($R_{\rho\tau\mu\nu}$)

In the linearized approximation, the Riemann tensor is the curl of the Christoffel connection, or the double gradient of h .

$$R_{\rho\sigma\mu\nu} \approx \frac{1}{2} (\partial_{\sigma} \partial_{\mu} h_{\rho\nu} + \partial_{\rho} \partial_{\nu} h_{\sigma\mu} - \partial_{\sigma} \partial_{\nu} h_{\rho\mu} - \partial_{\rho} \partial_{\mu} h_{\sigma\nu}) \quad (5)$$

We define a second-derivative helper, `doubleGradH`, that differentiates `metricGradient` again.

```

In[*]:= ClearAll[doubleGradH, RiemannLinear];

(*Acceleration → 0 for this model*)
fourGrad[dλds[ud_], _] := 0;

doubleGradH[σ_, ρ_, α_, β_] :=
Module[{firstDeriv},
  (*Instantiate the first derivative with correct indices*)
  firstDeriv = gradH[ρ, α, β];
  (*Differentiate w.r.t σ*)
  fourGrad[firstDeriv, σ] //. tensorRules];

RiemannLinear[D[ρ_], D[σ_], D[μ_], D[v_]] :=

$$\frac{1}{2} \left( \text{doubleGradH}[D[\sigma], D[\mu], D[\rho], D[v]] \right. \\
+ \text{doubleGradH}[D[\rho], D[v], D[\sigma], D[\mu]] \\
- \text{doubleGradH}[D[\sigma], D[v], D[\rho], D[\mu]] \\
\left. - \text{doubleGradH}[D[\rho], D[\mu], D[\sigma], D[v]] \right);$$


RiemannRaw = RiemannLinear[D[ρ], D[σ], D[μ], D[v]];
RiemannTensor = Collect[Expand[RiemannRaw], {r, k}, FullSimplify];

Print["Riemann  $R_{\rho\sigma\mu\nu}$  tensor (component extraction in the next section)"];
RiemannTensor /. dpyRules

Riemann  $R_{\rho\sigma\mu\nu}$  tensor (component extraction in the next section

```


Out[•]=

$$\begin{aligned} & \kappa \left(\frac{d\lambda_\sigma}{ds} (\eta_{\rho\nu} \lambda_\mu - \eta_{\rho\mu} \lambda_\nu) + \frac{d\lambda_\rho}{ds} (-\eta_{\sigma\nu} \lambda_\mu + \eta_{\sigma\mu} \lambda_\nu) + \frac{d\lambda_\nu}{ds} \eta_{\sigma\mu} \lambda_\rho - \frac{d\lambda_\mu}{ds} \eta_{\sigma\nu} \lambda_\rho - \frac{d\lambda_\nu}{ds} \eta_{\rho\mu} \lambda_\sigma + \frac{d\lambda_\mu}{ds} \eta_{\rho\nu} \lambda_\sigma \right) \\ & \quad \frac{2}{r^2} \\ & \quad \frac{3 \kappa (\lambda_\nu \sigma_\mu - \lambda_\mu \sigma_\nu) \left(-1 + \frac{d\lambda^\rho}{ds} \sigma_\rho \right)^2 (\lambda_\sigma \sigma_\rho - \lambda_\rho \sigma_\sigma)}{2 r^5} + \\ & \quad \frac{1}{2 r^3} \kappa \left(\eta_{\rho\nu} \lambda_\mu \lambda_\sigma - \eta_{\rho\mu} \lambda_\nu \lambda_\sigma + 2 \frac{d\lambda_\sigma}{ds} \lambda_\nu \lambda_\rho \sigma_\mu - \frac{d\lambda^\rho}{ds} \eta_{\sigma\rho} \lambda_\nu \lambda_\rho \sigma_\mu - \right. \\ & \quad 2 \frac{d\lambda_\rho}{ds} \lambda_\nu \lambda_\sigma \sigma_\mu + \frac{d\lambda^\rho}{ds} \eta_{\rho\rho} \lambda_\nu \lambda_\sigma \sigma_\mu - 2 \frac{d\lambda_\sigma}{ds} \lambda_\mu \lambda_\rho \sigma_\nu + \frac{d\lambda^\rho}{ds} \eta_{\sigma\rho} \lambda_\mu \lambda_\rho \sigma_\nu + \\ & \quad 2 \frac{d\lambda_\rho}{ds} \lambda_\mu \lambda_\sigma \sigma_\nu - \frac{d\lambda^\rho}{ds} \eta_{\rho\rho} \lambda_\mu \lambda_\sigma \sigma_\nu + \frac{d\lambda_\nu}{ds} \lambda_\mu \lambda_\sigma \sigma_\rho - \frac{d\lambda^\rho}{ds} \eta_{\rho\nu} \lambda_\mu \lambda_\sigma \sigma_\rho - \frac{d\lambda_\mu}{ds} \lambda_\nu \lambda_\sigma \sigma_\rho + \\ & \quad \frac{d\lambda^\rho}{ds} \eta_{\rho\mu} \lambda_\nu \lambda_\sigma \sigma_\rho - 2 \frac{d\lambda_\nu}{ds} \frac{d\lambda_\sigma}{ds} \sigma_\mu \sigma_\rho + 2 \frac{d\lambda_\mu}{ds} \frac{d\lambda_\sigma}{ds} \sigma_\nu \sigma_\rho + \eta_{\sigma\mu} \lambda_\nu \lambda_\rho \left(1 - \frac{d\lambda^\rho}{ds} \sigma_\rho \right) + \\ & \quad \left. \eta_{\sigma\nu} \lambda_\mu \lambda_\rho \left(-1 + \frac{d\lambda^\rho}{ds} \sigma_\rho \right) + \left(\frac{d\lambda_\nu}{ds} \left(-\lambda_\mu \lambda_\rho + 2 \frac{d\lambda_\rho}{ds} \sigma_\mu \right) + \frac{d\lambda_\mu}{ds} \left(\lambda_\nu \lambda_\rho - 2 \frac{d\lambda_\rho}{ds} \sigma_\nu \right) \right) \sigma_\sigma \right) + \\ & \quad \frac{1}{2 r^4} \kappa \left(\sigma_\rho \left(3 \frac{d\lambda_\sigma}{ds} (\lambda_\nu \sigma_\mu - \lambda_\mu \sigma_\nu) \left(-1 + \frac{d\lambda^\rho}{ds} \sigma_\rho \right) + \lambda_\sigma \left(3 \frac{d\lambda_\mu}{ds} \sigma_\nu + 3 \frac{d\lambda_\nu}{ds} \sigma_\mu \left(-1 + \frac{d\lambda^\rho}{ds} \sigma_\rho \right) + \right. \right. \right. \\ & \quad \left. \left. \frac{d\lambda^\rho}{ds} \left(-\lambda_\nu \lambda_\rho \sigma_\mu + \sigma_\nu \left(\lambda_\mu \lambda_\rho - 3 \frac{d\lambda_\mu}{ds} \sigma_\rho \right) \right) \right) \right) + \left(-3 \frac{d\lambda_\rho}{ds} (\lambda_\nu \sigma_\mu - \lambda_\mu \sigma_\nu) \left(-1 + \frac{d\lambda^\rho}{ds} \sigma_\rho \right) + \right. \\ & \quad \left. \lambda_\rho \left(-3 \frac{d\lambda_\mu}{ds} \sigma_\nu - 3 \frac{d\lambda_\nu}{ds} \sigma_\mu \left(-1 + \frac{d\lambda^\rho}{ds} \sigma_\rho \right) + \frac{d\lambda^\rho}{ds} \left(\lambda_\rho (\lambda_\nu \sigma_\mu - \lambda_\mu \sigma_\nu) + 3 \frac{d\lambda_\mu}{ds} \sigma_\nu \sigma_\rho \right) \right) \right) \sigma_\sigma \right) \end{aligned}$$

Intuitions

The raw output for $R_{\rho\sigma\mu\nu}$ above is a formidable “wall of math” comprising dozens of terms scaling from r^{-2} down to r^{-5} . You might find this alarming. However, **this verbosity is not a bug, it’s a feature** and primary virtue of $\mathcal{UD}$.

In a manual calculation, a physicist would usually discard higher-order terms and exploit symmetries in intermediate steps to keep the expressions manageable. While elegant, that process is perilous; a single dropped index or misplaced sign propagates errors that are extremely expensive to trace. By contrast, Wolfram’s symbolic engine is syntactically rigid. It does not simplify until told to do so, and it does not make algebraic mistakes. The “wall of math” is assurance that every interaction—every chain rule and every suitable contraction—is applied.

- *I have first-hand experience with this, leading me to cherish an ancestor of Mathematica/Wolfram. My first job out of school in the early 80's was to check the math for relativistic clock corrections in GPS signals, long before such was routine and automated. I recall doing page after page of algebra, only to fail a sign or index check at the end, then failing to find the bug on backtracking, and ending up starting over from scratch, multiple times. This very project caused me to seek out Stephen Wolfram, then at Caltech Physics, and get a copy of SMP from him to check those calculations. Without SMP, I would have had little confidence in my final results and would have felt irresponsible. Of course, SMP eventually became Mathematica, which became Wolfram, and here we are some 45 years later. I have used it every day for all kinds of tasks in science and engineering since long before it was a commercial*

product.

We can therefore trust this result *not* because the output is pretty or ugly or anything else, but because the process is robust. The same **fourGrad** and **tensorRules** that successfully reproduced the Liénard-Wiechert fields in electrodynamics applies here without modification.

Therefore, we do not need to read the raw tensor. Just as we write software to visualize large datasets, we will write a symbolic “Component Inspector” in the next section to probe this result. By projecting $R_{\rho\sigma\mu\nu}$ onto specific physical vectors, we will demonstrate that it collapses neatly into recognizable predictions of General Relativity.

That said, the big-picture structure is exactly what a relativist hopes to see:

- The Peeling Property: The terms sort themselves into powers of r . You clearly see the radiative $1/r$ terms (top line), the Coulomb/static r^{-2} terms, the near-field r^{-3} etc. terms.
- Evaluation Completeness: There are no “blue,” unevaluated symbols or calls of **fourGrad**. Every single term has been reduced to the fundamental physical variables: k , λ , σ , η , and $d\lambda/ds$.
- The Doppler Signature: The ubiquitous presence of the Doppler-like factor $1 - \frac{d\lambda^p}{ds} \sigma_\rho$ confirms that the retardation effects have been propagated through the second derivative.

Verification: The Component Inspector

Finally, we apply the following “Component Inspector” logic. We check the Newtonian limit (tidal tensor) and the radiation term.

```

In[*]:= (*NEWTONIAN TIDAL LIMIT (a → 0)*)
(*tidal tensor Eμν=Rμανβλαβ*)
Print["\n--- TEST 1: Newtonian Tidal Limit (a → 0) ---"];
(*Project onto velocity λ*)
tidalRaw = RiemannTensor /. {D[ρ] → D[μ], D[σ] → U[α], D[μ] → D[ν], D[ν] → U[β]};
(*Contract with λαβ*)
tidalContraction = tidalRaw * λ[D[α]] × λ[D[β]] //. tensorRules;
(*Limit: Acceleration → 0*)
newtonianLimit = tidalContraction /. {dλds[_] → 0};
newtonianLimit = Collect[newtonianLimit // FullSimplify, {r, k}];
Print["Tidal Tensor Eμν (Static Limit):"];
newtonianLimit /. dpyRules

(*RADIATION CHECK (1/r Term)*)
Print["\n--- TEST 2: Gravitational radiation term (1/r2) ---"];
(*Extract coefficient of 1/r2 *)
radTerm = Coefficient[RiemannTensor, r, -2];
If[radTerm === 0,
  Print["FAILED: No radiation term found."],
  Print["PASSED: Radiation term detected. Leading terms shown:"]];
(*Display snippet*)
Short[radTerm /. dpyRules, 5]

--- TEST 1: Newtonian Tidal Limit (a → 0) ---
Tidal Tensor Eμν (Static Limit):

```

$$\text{Out[*]} = -\frac{3 k \lambda_\alpha \lambda_\beta (\sigma_\mu \lambda^\alpha - \lambda_\mu \sigma^\alpha) (\sigma_\nu \lambda^\beta - \lambda_\nu \sigma^\beta)}{2 r^5} + \frac{k \lambda_\alpha \lambda_\beta (-\lambda_\mu \lambda_\nu \eta^{\alpha\beta} + (\lambda_\mu \eta^\alpha{}_\nu - \eta_{\nu\mu} \lambda^\alpha) \lambda^\beta + \lambda_\nu \lambda^\alpha \delta_\mu{}^\beta)}{2 r^3}$$

```

--- TEST 2: Gravitational radiation term (1/r2) ---
PASSED: Radiation term detected. Leading terms shown:

```

$$\text{Out[*]} // \text{Short} = \frac{1}{2} k \left(\frac{d \lambda_\sigma}{d s} (\eta_{\rho\nu} \lambda_\mu - \eta_{\rho\mu} \lambda_\nu) + \frac{d \lambda_\rho}{d s} (-\eta_{\sigma\nu} \lambda_\mu + \eta_{\sigma\mu} \lambda_\nu) + \frac{d \lambda_\nu}{d s} \eta_{\sigma\mu} \lambda_\rho - \frac{d \lambda_\mu}{d s} \eta_{\sigma\nu} \lambda_\rho - \frac{d \lambda_\nu}{d s} \eta_{\rho\mu} \lambda_\sigma + \frac{d \lambda_\mu}{d s} \eta_{\rho\nu} \lambda_\sigma \right)$$

Intuitions

The Newtonian Tidal Limit contains two distinct scalings:

$$E_{\mu\nu} \sim \frac{A}{r^3} + \frac{B}{r^5} \quad (6)$$

This is the signature of the tidal tensor—the gradient of the gravitational force. The $1/r^3$ term is the

trace part and the $1/r^5$ term is the traceless quadrupole part. In Newtonian gravitation, the tidal tensor is

$$\partial_i \partial_j \left(\frac{1}{r} \right) \sim (3 x_i x_j - r^2 \delta_{ij}) / r^5 \quad (7)$$

See that the $3 x_i x_j / r^5$ scales as $1/r^3$. The fact that $\mathcal{UD}$ found the relativistic version of this behavior after blindly differentiating $1/r$ twice is confirmation that $\mathcal{UD}$ finds the right geometry in the thicket of indices.

The radiation term, scaling as $1/r^2$, is proportional to acceleration $d\lambda/ds$. This is unlike the electro-dynamical radiation terms, which fall as $1/r$. However, it shows that the aberration effect is sensitive not only to velocity, as with electrodynamics, but to acceleration as well.

Finally, structures like $\frac{d\lambda}{ds} \eta \dots$ signal coupling between the acceleration vector and the space-time metric, η . This is the signature of a transverse gravitational wave.

Conclusion

We have extended the $\mathcal{UD}$ (up-down) calculus from electrodynamics to general relativity. By formalizing index valence and implementing differentiation rules for retarded quantities, we have overlaid a tensorial symbolic engine on top of Wolfram. $\mathcal{UD}$ can derive the linearized Riemann tensor from first principles.

$\mathcal{UD}$ overcomes the primary hurdle of manual relativistic calculations—“index explosion”. It handles differentiation of the retarded distance r , correctly resolving the peeling structure of the gravitational field into its Coulomb and radiative components.

$\mathcal{UD}$ is by now a proof-of-concept for a full-featured embedded tensor compiler on top of the Wolfram Language. Future work on the physics end will extend this framework to full non-linear General Relativity, where the metric $\eta_{\mu\nu}$ is replaced by the dynamic metric $g_{\mu\nu}$, letting us symbolically derive, for instance, the exact Schwarzschild and Kerr solutions. On the compiler end, alpha-renaming and index coalescing should completely eliminate the remaining, nagging manual interventions to prevent name collisions.